

AgentForge Studio

AgentForge Studio — The Whole System

AgentForge Studio turns a plain-English app idea into a specified, generated, tested and deployed Next.js + MongoDB web application, with a human watching every step in a browser studio and able to intervene at any point.

Repository	https://github.com/Ravindu200232/full-stack-ai-web-developer
Branch	main + all branches
Scope	The whole system: stack, tools, how the parts work together, and every repository it stands on.
Written	2026-08-14

Contents

Contents	2
1. Overview	3
2. What it does	3
3. Technology and tools	6
4. How it works	9
1. start.bat brings up two servers, in order	9
2. server.py's main() starts four threads and then blocks on the websocket	9
3. The studio loads and connects	9
4. An idea becomes an SRS project, with its attachments read first	9
5. The interview runs, one question at a time	9
6. A plan is written, shown, and approved	9
7. The SRS is generated, reviewed, revised and approved	9
8. The build starts and the SRS is adopted into the project folder	10
9. The architect writes plan.md and then every file, in one pass	10
10. Images, the plan audit, install, and the build fix loop	10
11. Tests are authored while the app generates, then run and repaired	10
12. The dev server comes up and the runtime repair loop runs	10
13. Security, performance, and one report written to disk	10
14. Deployment: analyze, review, deploy, adopt	11
5. The code, file by file	12
6. Why it is built this way	14
7. Interfaces	16
8. Repositories	18
Open source it is built on	18

1. Overview

AgentForge is an app builder that runs on your own machine. You type an idea; a guided interview turns it into a written specification with diagrams and a PDF; a model then writes an entire Next.js 16 + Tailwind + MongoDB application file by file; generated Vitest unit tests and Playwright end-to-end flows are run against it and their failures fed to a repair loop; and one button deploys the result to Vercel, AWS EC2 or AWS ECS Fargate with the infrastructure generated as CloudFormation and the image built inside GitHub Actions. Every model call goes to Ollama — a local daemon, or ollama.com with a key the user pasted in. Deployment is the only step that talks to anybody else, and only to accounts the user connects.

The problem it solves is the gap between the four things that normally live in four different products. A specification tool does not build. A code generator does not test. A test runner does not deploy. A deployment dashboard has never read the spec. AgentForge chains them, and — more importantly — makes each one hand the next something it can act on: the approved SRS becomes the architect's brief, the architect's `plan.md` becomes what the analyzer audits the finished app against, the QA manifest maps each failing test back to the one app file a fix may touch, and the deployed run's record is copied back into the project folder so reopening it a week later still shows what was deployed and where.

At runtime it is two operating-system processes and six listening ports. `python server.py` is one process that starts four threads: the HTTP API on 7824, a websocket server on 7825, the SRS agent's FastAPI app under uvicorn on 7826, and the deployment agent's own `ThreadingHTTPServer` on 7834. `npm run dev` in `studio/` is the second process, Next.js 16 on 3000 under a `basePath` of `/\_\_agentforge`. A seventh listener comes and goes: the generated app's own `next dev` on 5173, which the studio proxies so the preview iframe is same-origin with the studio — which is what makes click-to-edit and draw-on-the-page possible at all.

The repository is one tree with a branch per component. `main` and `srs-review-and-revision` (the work branch) hold everything; `srs-agent`, `builder-agent`, `qa-agent` and `deployment-agent` each hold only that subsystem's paths. The SRS agent and the deployment agent are verbatim copies of two standalone products, and they are meant to stay copies: everything AgentForge-specific about each is collected in a single `bridge.py`, so a future re-sync is a directory copy rather than a merge. That constraint is visible in the code — `srs\_agent/app/config.py` says outright that it is the one file in its tree that differs, and the deployment copy renames `agents/` to `dfagents/` and `server.py` to `dfserver.py` only because AgentForge already owns those names at the root of `sys.path`.

2. What it does

Feature	What it does	Where
The idea box, with worked examples	The studio opens on a single prompt box with three fully-written example ideas (a darkroom co-op, a bike workshop, a small hotel) that fill it in one click. Two ways forward: build straight away, or "plan first" to run the interview and produce an SRS before a line is written.	studio/components/Home.jsx, studio/app/page.jsx
Attach a picture, a document or a recording to the idea	A photo, a PDF price list or a spoken note can be attached before anything is planned. They are uploaded before `analyze` runs, not after, because the questions are generated from the brief and the brief is the idea plus every source — attaching the price list afterwards means being asked what is on it.	studio/components/srs/Attachments.jsx, studio/lib/use-attachments.js, srs-agent/srs_agent/app/extraction/

Feature	What it does	Where
A guided interview, one question at a time	The SRS agent asks short, plain-language questions with three to five concrete options each. Question N is genuinely conditioned on answers 1..N-1 — the model is handed the raw idea, every answer so far and the app-type pack — but the queue, the conditions and the answer storage are deterministic, so a bad model response costs one clumsy question, never a broken interview.	srs-agent/srs_agent/app/agents/interview.py, srs-agent/srs_agent/app/knowledge/topics.py, studio/components/srs/Interview.jsx
A one-page plan to approve before the spec exists	Before the SRS is written the customer approves a plain-English plan: who uses it, what screens exist, what it keeps track of, what it does. The plan is a ceiling — the specification's tables come from its `records`, its roles from its `users`, its pages from its `screens` — so a record left out here cannot be added later.	srs-agent/srs_agent/app/agents/plan_generator.py, srs-agent/srs_agent/app/services/plan_approval.py, studio/components/srs/PlanReview.jsx
Revise the specification by typing at it	The generated SRS can be changed in conversation — "add a waiting list", "members should not see prices" — and each revision returns a diff summary and a new version number you can look back at. Approving it produces the builder handoff prompt.	studio/components/srs/SrsReview.jsx, srs-agent/srs_agent/app/routers/customize.py, srs-agent/srs_agent/app/agents/customization.py
A logo you generate or one you already own	With image generation switched on, the build pauses on a logo panel: a prompt is written for you, Fooocus draws it, and you accept, redraw or skip. Uploading your own PNG takes the same path and produces the same answer, so the build cannot tell which one drew it.	studio/components/LogoPanel.jsx, agents/images.py, server.py (/api/image, /api/image-upload)
Watch the app being written, file by file	The Code tab streams each file token by token as the model writes it, and the studio jumps to it automatically on the first stream. The Pipeline rail shows the plan's phases and their status alongside.	studio/components/CodePanel.jsx, studio/components/Pipeline.jsx, studio/lib/ws.js
A live preview at three sizes, with its own history	The finished app runs in an iframe at desktop, tablet (834px) and mobile (390px). Back and forward are tracked in the studio rather than through `contentWindow.history`, because an iframe shares the joint session history and `history.back()` on a preview that has not navigated yet steps the whole studio backwards.	studio/components/PreviewPane.jsx
Point at something in the preview and change just that	Click the picker, click a heading or a button, type what should change. The clicked element's tag, text, classes, ancestors and current route are resolved to the one file that renders it — the route narrows a six-way text match to one before a single token is spent — and the edit is guarded so a "make this teal" cannot become a page rewrite.	studio/lib/picker.js, agents/picker.py, server.py (run_element_edit)
Draw over the page and describe the change	The pencil tool lets you scribble a region on the preview and say what you want there. The screenshot is taken server-side with Playwright at the iframe's exact CSS size, signed in with the demo credentials first, with the strokes drawn into the page — because no browser can rasterise an iframe from the parent document.	agents/capture.py, server.py (run_pencil_edit), studio/components/PreviewPane.jsx
Swap a picture for your own	Point at any image in the preview and upload a replacement. It goes over HTTP rather than the websocket — a multi-megabyte base64 frame is not what that socket is for — but the work still runs in a thread and reports over the socket, so the log, the phases and the undo point arrive exactly as they do for every other edit.	server.py (run_image_swap, /api/image-swap), studio/lib/api.js
Undo the last edit	Every whole-file rewrite saves the previous bytes first and emits an undo point naming the files. The snapshots live outside the project directory on purpose — the project is walked to build the export zip, so an in-project undo folder would ship to the user.	server.py (_snapshot, restore_snapshot, /api/undo), qa_agent/snapshot.py

Feature	What it does	Where
A nine-tab testing report, exportable as a PDF	Overview, Unit Testing, Test Timeline, Integration, API Contracts, Bug Reports, Security, Performance and the test sources. The PDF is rendered on request rather than written at the end of the build, because the tab keeps changing as tests are repaired and a PDF written once would be stale the first time somebody fixed a test.	studio/components/testing/TestingResult.jsx, qa_agent/report_pdf.py, server.py (read_qa_results, /api/qa-pdf)
A nine-tab SRS view over the spec the app was built from	Document, Approved Plan, Requirements, Data, Roles & Access, Handoff, Diagrams, Interview and Risks — read back out of the project folder the SRS was adopted into, with the IEEE-830 PDF one click away.	studio/components/srs/views.jsx, studio/components/srs/SrsResult.jsx, server.py (read_srs_results, /api/srs-pdf)
One-button deployment to three targets	Vercel, AWS ECS Fargate behind a load balancer, or a single AWS EC2 instance with an Elastic IP, nginx and systemd (port 22 shut, administration over SSM). Each target's blurb says what it costs. The studio drives an analyze → review → deploy autopilot and reports the agent's own events into the build log.	studio/components/deploy/DeployPanel.jsx, studio/lib/deploy-constants.js, server.py (start_deployment, _deploy_autopilot)
Ten monitor tabs over the live deployment	Deploy, Overview, Pipeline, Repository, CI/CD, Infrastructure, IAM & Security (AWS only), Logs, API Validation and Evidence — each stage of the target's pipeline named with what it does, and a masked evidence bundle you can export as a ZIP or a PDF.	studio/components/deploy/MonitorViews.jsx, studio/components/deploy/MonitorPipeline.jsx, deployment-agent/deploy_agent/dfserver.py
Cancel a deployment, or destroy what it built	A run can be stopped mid-flight and its cloud resources torn down. The project's deployment record is archived to <code>`.agentforge/deploy-archive/<run_id>`</code> rather than deleted — the event stream, the readiness score and the last monitor snapshot exist nowhere else once the stack is gone.	studio/components/deploy/DeployDanger.jsx, server.py (retire_deploy, _deploy_marker)
Pick a different model for each job	Separate pickers for the build agent, the QA author, the SRS and the deployment plan, over whatever Ollama has locally plus the cloud catalogue. QA and the SRS refuse to run on a local model and say so plainly rather than producing a worse result quietly.	studio/components/ModelPicker.jsx, studio/lib/models.js, agents/ollama_client.py, srs-agent/srs_agent/bridge.py
Download the generated project as a ZIP	The whole project is zipped in the browser from the files already in the store, so no server round-trip is needed to hand somebody the code.	studio/app/page.jsx (downloadZip), studio/package.json (jszip)

3. Technology and tools

Technology	Version	What it does here	Why
Python	3.11+ (start.bat refuses below it)	The spine: server.py, the builder agents, the QA layer, and the two mounted agents all run in one interpreter.	Every long-running tool the pipeline drives — npm, Playwright, boto3, gh, mongod — is easiest to supervise from a process that owns them.
http.server ThreadingHTTPServer	—	The HTTP API on :7824, and the deployment agent's own server on :7834.	No framework dependency for the spine, and a thread per request means a blocking <code>`npm ci`</code> or Playwright call stalls only its own request. <code>`_UIServer`</code> overrides <code>`handle_error`</code> because one measured feature run produced 424 tracebacks from iframe requests cut off mid-write.
websockets	>=13,<17	The build channel on :7825 — logs, phases, per-file token streams, test results, undo points.	The build talks continuously for minutes; polling would be the wrong shape. <code>`emit()`</code> marshals onto the loop with <code>`run_coroutine_threadsafe`</code> from whatever thread produced the event.
requests	>=2.31	Every inter-process call: the preview proxy, the SRS proxy, the deployment proxy, the job shims, Fooocus, and the live-site probe.	Streaming responses with <code>`r.raw.stream(65536, decode_content=False)`</code> is what lets the proxies relay bodies byte for byte without buffering.
Next.js	^16.3 (studio); 16.3.0 pinned in generated apps	The studio UI, and the framework of every application AgentForge generates.	For the studio, <code>`basePath: '/__agentforge`</code> plus two rewrites put the API and the generated app on one origin — which is what makes the preview iframe same-origin and the element picker legal. For generated apps, the installed package carries its own 444-file documentation set that the model can read on demand.
React	^19.2 (studio); 19.0.8 in generated apps	The studio's components, and the generated apps'.	
Tailwind CSS	^4.3 (studio) via @tailwindcss/postcss; ^3.4.13 (generated)	Styling in the studio and in every generated app.	The generated stack pins Tailwind 3.4.13 with a Python-written config, because a config the model writes wrongly breaks everything else it writes.
Zustand	^5	The studio's client state — project, files, logs, phases, test counts, models, SRS phase — persisted to localStorage under a KEYS map.	The websocket handler is a plain <code>`switch`</code> writing into one store; nothing needs a provider tree.
JSZip	^3.10	Packaging a generated project for download, in the browser.	The files are already in the store, so no server round-trip is needed.
FastAPI	—	The SRS agent's HTTP surface: eight routers covering projects, analyze, interview, plan, srs, customize, downloads and events.	Deliberately unpinned in srs-agent/requirements.txt — the standalone product pinned fastapi==0.115.0, and reinstating that would downgrade the version AgentForge itself already runs on to satisfy a constraint the code does not have.

Technology	Version	What it does here	Why
Uvicorn	—	Serves the SRS app on :7826 inside AgentForge's process, on its own daemon thread.	<code>Server.run()</code> calls <code>asyncio.run()</code> , so it brings its own loop — which is exactly right here, because AgentForge's websocket server already owns the main thread's loop. <code>timeout_keep_alive</code> is raised to 300s for the event stream.
LangGraph + langchain-core	—	Three compiled graphs: analysis (intake → clarify classify), generation (audit → generate_srs → diagrams), and customization (customize → diagrams).	Each graph maps to a phase separated by human input — answering questions, approving the plan — which keeps every one of them a clean DAG. The interview is deliberately NOT a graph node: it is one question at a time, driven by the customer.
Pydantic + pydantic-settings	—	The SRS's request/response schemas and its settings object.	<code>Settings.model_post_init</code> is where the AgentForge values land, imported from <code>bridge</code> rather than read from environment variables — an env var only works if it is set before the module is first imported, and nothing can enforce that from a package importable from a server thread, a test, or a scratchpad script.
httpx	—	The SRS agent's Ollama calls, and its job runner's in-process ASGI replay.	<code>httpx.ASGITransport(app=_app)</code> is what lets a queued job be replayed straight into the FastAPI app — no socket, no second port, and the eight routers never learn the shim exists.
Motor + PyMongo	pymongo[srv]>=4.9, <5	The SRS store (<code>agentforge_srs</code>), with an in-process dict store as the fallback backend.	Routers and agents depend on a <code>Store</code> protocol only, so the fallback is transparent — the whole app runs in CI with no Mongo server. A middleware re-checks the connection every five seconds because mongod is being started in another thread at the same moment this app first connects.
Ollama	—	Every model call in the system: the architect, the analyzer, the bug fixer, the QA author, the SRS, the deployment planner, the intent router.	A model is treated as cloud when its name ends <code>-cloud</code> or <code>:cloud</code> . Two routes are supported — the local daemon proxying after <code>ollama signin</code> , or a direct bearer token to ollama.com — and a configured key wins. The SRS side routes through AgentForge's client rather than talking to Ollama itself, because the copied adapter posted every model to localhost:11434.
MongoDB (vendored mongod)	27017 by default	The database every generated app uses, and the SRS's store.	AgentForge's promise is that nothing has to be installed or signed up for, so it vendors <code>mongod</code> the way it vendors Node. Only the one binary is fetched: the Windows ZIP is 923 MB, 91% debug symbols, so the ZIP central directory is read over HTTP byte ranges and the single 32.5 MB member inflated — 37.7 MB transferred instead of 922.6 MB. An already-listening mongod is adopted, never killed.

Technology	Version	What it does here	Why
Foocus (driven over its Gradio endpoint)	—	Generating logos, hero images and seeded product photos into a project's <code>public/generated/</code> .	Foocus ships no REST API and no <code>api_name`</code> on any handler, and its generate handler takes 153 positional arguments. So the argument list is read from Foocus's own <code>foocus_config.json`</code> and four values are overridden — prompt, aspect, count, seed. <code>image_host`</code> can point at another machine, because the workstation with the GPU is not always the one running AgentForge.
Playwright (Python API)	>=1.48,<2	The route/render tester, the end-to-end sign-in pass, the pencil tool's region screenshot, and the browser Lighthouse drives.	Python-only, so there are no JS test files, no <code>require()`</code> and no ES-module problems. Measured: <code>playwright==1.60.0`</code> wants chromium 1223, which is already in <code>~/AppData/Local/ms-playwright`</code> , so an e2e run costs no npm install and no browser download — <code>npx playwright test`</code> would cost ~50 MB per project.
Vitest	—	The generated unit test suites.	AgentForge writes the config and the mocks; the model writes only the test bodies. <code>jsconfig.json`s `@/*` alias is not read by Vitest, `vi.mock` is hoisted above imports, and these projects have no <code>"type": "module"</code> so the config must be <code>.mjs`</code> — every one of those is a configuration failure that reads exactly like an application bug.</code>
ReportLab	>=4.2,<5	The SRS PDF, the test report PDF, the deployment report, and this documentation set.	Pure Python, so it works on Windows with no native libraries. It also draws the SRS diagrams as vector graphics for print, so the PDF always contains real diagram images even when mermaid-cli cannot be found.
pypdf, PyMuPDF, pytesseract, Pillow, faster-whisper	—	Reading attachments: a PDF's text layer, a rendered page for the vision model when there is none, OCR on an image, and transcription of a recording.	The PDF decision is made per page, not per file — an exported invoice has an exact text layer and a scan has none, and a customer who attaches their entire requirements document as a scan would otherwise teach the intake nothing. Each is optional and each degrades to a structured setup note rather than a crash.
boto3 / botocore[crt]	>=1.34,<2	Every AWS call the deployment agent makes: CloudFormation, EC2, ECR, ECS, S3, IAM, Secrets Manager, SSM, SSO onboarding.	
GitHub CLI + GitHub Actions, CloudFormation, Vercel CLI/API	—	The delivery path: the reviewed artifacts are committed to a GitHub repository, CI validates them, and the workflow builds the image or the prebuilt output and rolls it out. Infrastructure is generated CloudFormation.	The container image is built on the runner, never on this machine — nothing needs Docker installed locally. Secrets go to AWS Secrets Manager or Vercel's production environment, never into GitHub.

4. How it works

1. start.bat brings up two servers, in order

It checks python and node (the two it cannot install for you), installs git, gh, aws, ollama through winget and vercel through npm only if they are missing, pulls an Ollama cloud model if none is present, then starts `python server.py` and waits for :7824 to listen before starting `npm run dev` in studio/ and waiting for :3000. The order matters: the studio proxies straight through to Python, so a studio that comes up first answers 502. A port already in use is reused, not restarted.

2. server.py's main() starts four threads and then blocks on the websocket

`MONGO.ensure_running`, `start_srs_api` (uvicorn on 7826), `start_deploy_api` (ThreadingHTTPServer on 7834) and `start_http` (the API on 7824) each get a daemon thread; the main coroutine then holds `websockets.serve(ws_handler, bind_host(), 7825)` open forever. Both mounts have two separate try blocks — an import failure and a runtime failure are different problems — and neither may propagate: a broken agent is a tab that explains itself, never an AgentForge that will not start.

3. The studio loads and connects

`studio/app/page.jsx` hydrates the Zustand store from localStorage, calls `connect()` in `lib/ws.js` to open `ws://host:7825`, and fetches `/projects` and `/models`. Every API call goes to `/__agentforge/api/...` on the studio's own origin; `next.config.js` rewrites that to `127.0.0.1:7824` and everything else to `127.0.0.1:5173`. If the socket is down, `send()` falls back to the matching HTTP route in `HTTP_FALLBACK`.

4. An idea becomes an SRS project, with its attachments read first

"Plan first" saves the chosen SRS model, POSTs `/srs/projects` with the idea, uploads any attachments to `/projects/{id}/inputs-json`, then POSTs `/analyze`. Attachments go up before analyze because the questions are generated from the brief and the brief is the idea plus every source. The analysis graph runs `intake` → `classify` (or → `clarify`, when the idea reads as nonsense) and produces the question queue.

5. The interview runs, one question at a time

`GET /projects/{id}/interview` returns the next question and `POST .../interview/answer` records it. `InterviewAgent` is given the raw idea, every answer so far and the app-type pack before each question, so question N is conditioned on answers 1..N-1; the queue and the conditions come from `knowledge/topics.py` and are deterministic. The model supplies wording and options only.

6. A plan is written, shown, and approved

`PlanGeneratorAgent` composes a deterministic skeleton from the interview answers, then an LLM pass rewrites it in the customer's own terms — any failure keeps the skeleton, so the plan screen always has something to approve. `plan_approval` refuses a superseded version and refuses one with an unanswered required question, and `state()` returns the same verdict the approve endpoint would, so the button is never offered for something that will be refused.

7. The SRS is generated, reviewed, revised and approved

The generation graph runs `audit` → `generate_srs` → `render_diagrams`. The studio shows the document across nine tabs; typing a revision calls `/customize`, which reruns the customization graph and returns a diff summary and a new version. `POST /approve` then makes `/builder-handoff` available — a prompt capped at 1200 words and 8000 characters, a cap inherited from a different builder's 8192-token reply limit.

8. The build starts and the SRS is adopted into the project folder

The studio sends `{type:'agent_build', prompt, model, qa_model, logo, srs_id}` over the websocket; `run_agent_pipeline` names the project directory from the SRS's own app name, copies the approved logo to `public/logo.png`, and calls `adopt_srs`. That copies the staging folder from `production-ready/.srs/<id>/` and fetches four things the SRS only writes on demand — the plan, the handoff, the interview and the PDF. The staging copy is deliberately not removed, so building twice from one SRS keeps working.

9. The architect writes plan.md and then every file, in one pass

`_srs_name_line` puts the customer's chosen app name FIRST, above the handoff prompt — measured, an architect handed the name inside the detail section still shipped a different one — and `_srs_brief` appends the requirements, data, roles, access matrix, workflows, acceptance criteria and validation rules underneath, sized against the model's real context window and saying what it dropped. The model then writes files through a streaming `<write_file path="...">` protocol, told "Continue" whenever it stops, until every planned file is on disk. Config files are always Python-generated.

10. Images, the plan audit, install, and the build fix loop

`run_image_stage` and `_fill_missing_images` draw whatever the pages ask for; `AnalyzerAgent.run()` compares the finished tree against `plan.md` (file paths in Python — measured 0 false positives across nine projects — meaning by the model, credentials by a real HTTP request); `ensure_node_deps` installs; and `run_build_fix_loop` runs `npm run build` and repairs what it reports. `bundler_flag` adds `--webpack` on Next 16+ only, pinning the diagnostic vocabulary that three separate matchers depend on.

11. Tests are authored while the app generates, then run and repaired

`QASession` wraps the architect's existing `on_phase` callback — the architect never waits and `arch.convo` is never touched — and a second model call writes unit tests per phase. The unit stage runs Vitest through a Python-written `.mjs` config; the e2e stage has the model write a declarative FLOW/GOTO/FILL/CLICK scenario, never code, which maps to fixed Python calls and to an emitted `.spec.js` generated from the same parsed form. `agents/bugfixer.py` arbitrates first — it must emit `VERDICT :: test|code` before any write block, and the verdict decides the write allowlist.

12. The dev server comes up and the runtime repair loop runs

`start_next` launches node against `node_modules/next/dist/bin/next` directly, and the Playwright tester probes every route. Errors from the browser and from the dev server's own stream are merged; Next 16.2+'s `/_next/mcp` `get_errors` is read while Playwright has a browser open, because that tool needs a browser session. The loop stops on three conditions: no errors, `MAX_FIX=6` attempts, or a 900-second budget — and also when an attempt changed none of the previous errors, because further attempts would repeat it.

13. Security, performance, and one report written to disk

`run_security_stage` runs `npm audit` and the deterministic code checks, snapshots the files before repairing blockers and majors, and reverts the fix if it broke the build — the finding then stands. `run_perf_stage` drives Lighthouse through the Chromium Playwright already downloaded and labels the result `measured on next dev` — not a production build. `write_qa_report` collects everything into `agentforge/qa/report.json`, with `ran: false` on any stage that did not.

14. Deployment: analyze, review, deploy, adopt

`\_deploy\_autopilot` POSTs `/api/runs/analyze` to the agent, waits for REVIEW_READY, refuses to continue if the plan carries no `model\_used`, then POSTs `/deploy` with the production MongoDB URI (a loopback URI is rejected rather than written into Secrets Manager as a production database). It drains the agent's events by cursor into the build log — the agent's own websocket on :7835 was not ported because the same events are in its SQLite with an id, and a cursor read is replayable after a reload. After 240s parked in VALIDATING it asks the site itself, because the agent's readiness score has a ceiling of 85 in some configurations and can never reach the 90 it needs. `adopt\_deploy` then copies the record into the project.

5. The code, file by file

Path	What lives there
server.py	8,009 lines and the spine of the whole system: the HTTP API and its proxies, the websocket handler, <code>`run_agent_pipeline`</code> , every editing tool, the image endpoints, the two job shims, the SRS and deployment glue, and the process lifecycle.
start.bat	The launcher. Installs what is missing, pulls a cloud model if there is none, starts the backend and waits for :7824, starts the studio and waits for :3000, then opens the browser. <code>`start.command`</code> is the macOS twin.
Requirements.txt	The root dependency list — and the deployment agent's too, since that agent runs in this interpreter. Comments record what is deliberately absent and why.
studio/next.config.js	<code>`basePath: '/'_agentforge`</code> plus two <code>`beforeFiles`</code> rewrites: the API to 127.0.0.1:7824, and everything else to the generated app on 127.0.0.1:5173. This one file is what makes the studio and the preview share an origin.
studio/lib/api.js	Every HTTP call the studio makes, and the three job-shim pollers (<code>`localJob`</code> , <code>`srsJob`</code> , <code>`deployJob`</code>). Also <code>`MAX_UPLOAD_BYTES = 7_500_000`</code> , measured against the rewrite's 10 MiB body limit less base64's one-third inflation.
studio/lib/ws.js	The websocket connection, a 3-second reconnect, a <code>`switch`</code> mapping 20-odd message types onto store actions, and <code>`send()`</code> with its <code>`HTTP_FALLBACK`</code> route table for when the socket is down.
studio/lib/store.js	The Zustand store: project, files, logs, phases, live test counts, models, think flag, SRS phase and id — with the persisted keys and the hydrate/reset logic.
studio/app/page.jsx	The studio shell: the five workspace tabs (SRS, Preview, Code, Testing, Deploy), the ask box, open/resume/import/zip, and the home-versus-workspace screen switch.
studio/components/Home.jsx	The first screen: the idea box, three worked examples, attachments, and the two paths — build now, or plan first through the SRS phases (planning → interview → plan → review).
studio/components/PreviewPane.jsx	The preview iframe at three viewport sizes, its own navigation trail, and the three in-page edit tools: the element picker, the pencil, and picture swap.
studio/components/testing/TestingResult.jsx	The nine testing sub-tabs and the PDF export button.
studio/components/srs/views.jsx	The nine SRS sub-tabs and their badge logic, shared by the review screen and the project's SRS tab.
studio/components/deploy/DeployPanel.jsx	Target selection, account connection cards (GitHub, Vercel or AWS, MongoDB), and the ten monitor views over a live run.
srs-agent/srs_agent/mount.py	Runs the SRS FastAPI app under uvicorn on a daemon thread, attaches the job router, and installs a middleware that re-checks the Mongo connection every five seconds — because mongod is being started in another thread at the same moment.
srs-agent/srs_agent/bridge.py	Every seam between the SRS agent and AgentForge in one file: the repo root, the staging directory, the port, the model choice, Ollama routing, the context window, and the Mongo URI. <code>`app/config.py`</code> is the only file in the copied tree that imports it.
srs-agent/srs_agent/jobs.py	The SRS job shim. Records the measurement that forced it: <code>`generate-srs`</code> returned 200 in 84.9s straight to Python and 500 at exactly 30.0s through the rewrite. Replays queued requests in-process over <code>`httpx.ASGITransport`</code> .
srs-agent/srs_agent/app/main.py	The FastAPI application: lifespan connects and closes the store, CORS, eight routers, and a <code>`/health`</code> that reports the store backend and Ollama's reachability.
srs-agent/srs_agent/app/graph/workflow.py	The three compiled LangGraph graphs, and the note explaining why the interview is not one of them.

Path	What lives there
deployment-agent/deploy_agent/mount.py	Runs the deployment agent's own ThreadingHTTPServer on a daemon thread, sets its data directory and no-browser flag, and kicks off run reconciliation. Explains why this one keeps its own server instead of becoming ASGI routes.
deployment-agent/deploy_agent/bridge.py	The deployment agent's single seam file, including the two renames — `agents/` → `dfagents/` and `server.py` → `dfserver.py` — and the reason for each.
deployment-agent/deploy_agent/dfs_server.py	The deployment API on :7834: health, onboarding, MongoDB check, AWS SSO, `runs/analyze`, and every per-run action (events, artifacts, diff, monitor, evidence, export.zip, report.pdf).
agents/architect.py	4,581 lines. The LLM-driven generator: raw prompt → `plan.md` → every file, in one uninterrupted pass, through a streaming write_file protocol. Owns both output stacks (Next 16 + MongoDB for new builds, Vite + React for projects made before the migration) and the Python-written scaffold.
agents/analyzer.py	The post-generation audit that reads the finished project back and compares it against the plan — file paths deterministically, meaning by the model, credentials by a real HTTP request — and repairs what it finds. Its `repair` call is reused by the security, syntax, import and route paths.
agents/mongo.py	The vendored mongod: resolve, or fetch just the one binary over HTTP byte ranges, run it on 127.0.0.1:27017, adopt an existing server rather than kill it, and hand each project its own database.
agents/ollama_client.py	The unified model client — local daemon and Ollama Cloud, the `~/.cloud/:cloud` routing rule, per-model context windows, the settings file at `~/.agentforge/settings.json`, and the catalogue the model pickers render.
qa_agent/session.py	The shell that lets tests be written while the app is still generating, by wrapping the one `on_phase` callback the architect already fires. Guarantees the architect never waits and the build conversation is never written to.

6. Why it is built this way

Design decisions with their reasons. Where the source records the failure that forced a choice, that failure is given rather than a general principle.

Decision	Why
The SRS agent and the deployment agent run inside server.py's process, each on its own daemon thread and its own port, rather than as separate services.	They are copies of two standalone products and are meant to stay copies. Mounting them in-process means the studio needs one backend address, both agents can reach AgentForge's Ollama routing and its mongod, and neither has to be re-architected. Nothing in either mount may raise into the caller: a broken agent is a tab that explains itself, never an AgentForge that will not start.
Two mounts, two shapes: the SRS runs under uvicorn on a fresh asyncio loop, the deployment agent keeps its own ThreadingHTTPServer.	The deployment agent's long work is boto3, `gh`, `npm ci` and Playwright, all blocking. Under uvicorn a single synchronous call stalls every other request on the loop — the bug that produced the SRS mount's comments was ReportLab doing exactly that. A ThreadingHTTPServer gives each of those calls its own thread, so rewriting thirty-five handlers onto a loop would have put the problem back.
Everything AgentForge-specific about a copied component lives in exactly one `bridge.py`, imported as a module rather than configured through environment variables.	A future re-sync is then a directory copy, not a merge — `srs_agent/app/config.py` says outright that it is the one file in its tree that differs. And an env var only works if it is set before the module that reads it is first imported, which cannot be enforced from a package importable from a server thread, a test, or a scratchpad script. An import has no ordering to get wrong.
Every call that can outlive 30 seconds becomes three fast ones: start a job, poll it, read the result. Implemented three times — for the local API, for the SRS, and for the deployment agent.	Measured, not assumed: `generate-srs` for a small school library returned 200 in 84.9s straight to Python and 500 at exactly 30.0s through Next's rewrite. The shim is deliberately generic rather than one per slow endpoint — a single interview answer was measured at 23.3s on a fast cloud model, and every LLM call scales with whichever model the user picked, so an endpoint-by-endpoint list would just be the ones that happened to be slow that day.
Uploads travel as base64 inside a JSON body, capped at 7.5 MB in the browser.	The job shim is what keeps a slow upload alive, and the shim replays JSON only — so the file has to be JSON, not multipart. The cap is measured against the running stack: the rewrite in front of both APIs refuses a body over 10 MiB with a bare 500 that never reaches Python, so nothing server-side could turn it into a sentence. base64 costs a third on top. Checking in the browser makes the answer instant and specific.
The studio and the generated app share one origin, with the studio under `/_agentforge` and everything else rewritten to :5173.	The preview iframe has to stay reachable and same-origin: reading its DOM is what the element picker does, and relaying its HMR websocket byte-for-byte is what stops every element edit from forcing a full iframe reload that throws away scroll position, form state and the logged-in view — exactly the state those tools operate on. It is also why the SRS could not solve its 30s problem the way the standalone app does, by bypassing its own proxy.
Every read-back endpoint returns a `have` map, and a stage that never ran is reported differently from a stage that ran and found nothing.	`read_qa_results` and `read_srs_results` both say it in as many words: "no ambiguities were found" and "the ambiguity stage never ran" are different facts, and a dashboard showing an empty list for both is lying about one of them. The pattern runs all the way through — `security_ran`, `unit['ran']`, `e2e.ran`, the PDF's "this stage did not run" block, and the Testing tab's "never ran" counters.
Every tool that edits an existing project funnels through one `verify_after_edit`.	`run_agent_update` used to ship whatever the model wrote and restart the dev server — no compile check, no request to any route — so "the model said it changed three files" and "the app still works" were the same claim. The funnel checks syntax, then named imports, then the build, then probes every route, and repairs at each step through the analyzer's existing repair call.
An automated fix that breaks the build is reverted, and the finding it was fixing stands.	In the security stage the files are snapshotted before repair, and if `run_build_fix_loop` cannot get green afterwards the snapshot is restored and the finding is reported unfixed. A described flaw and a fixed one are not the same deliverable, but a broken app is worse than either.

Decision	Why
Failures degrade with a stated reason instead of crashing or going silent.	<code>`_guarded`</code> turns any handler exception into a JSON body with the message, because without it <code>`BaseHTTPRequestHandler`</code> sends a 500 with an EMPTY body — measured on the logo panel, where a failed draw said "HTTP 500" and nothing else. <code>`srs_status`</code> and <code>`deploy_status`</code> return both what the thread believes and whether the port is measurably listening, because those disagree exactly when it matters. <code>`nextdocs`</code> returns "" on every error path, because a build must never fail because a documentation site was unreachable.
<code>`--webpack`</code> is passed to <code>`next dev`</code> on Next 16+ and never before it.	Next 16 made Turbopack the default, and Turbopack words its diagnostics differently from the <code>`Failed to compile`</code> / <code>`Module not found`</code> / <code>`Can't resolve`</code> strings that <code>`run_build_fix_loop`</code> , <code>`next_stderr`</code> and the tester all match on — pinning the bundler pins that vocabulary. But Next 15.5.22 has no <code>`--webpack`</code> flag at all and <code>`next dev`</code> exits immediately on an unknown argument, so passing it unconditionally would kill the dev server of every project already on disk.
A new build never writes into a directory that already holds an app.	The old rule made "a Next.js app is already here" the condition for reuse rather than avoidance. Since the folder name comes from the first forty characters of the prompt, three prompts sharing an SRS preamble landed in one folder — a POS app, a school app and a car dealership superimposed, 66 route URLs on disk against a 31-file plan. The build never noticed because <code>`arch.files`</code> starts empty; the analyzer did, and wrote nine route handlers for two dead applications.
The servers bind to loopback unless <code>`lan_access`</code> is switched on.	AgentForge's HTTP surface can start builds, edit files and run npm — none of which belongs on an open port by accident. Widening it is a deliberate setting, and it exists for one measured use: a second machine running AgentForge borrowing this one's GPU through <code>`/api/image`</code> .

7. Interfaces

Name	Shape
ws://127.0.0.1:7825	Inbound: build, agent_build, agent_resume, chat, agent_update, pencil_edit, element_edit, image_edit, feature, update. Outbound: log, step, progress, phase, file, stream_start/stream/stream_end, test_start/test_run/test_result/test_fixing, done, error, detected, chat_intent, agent_msg, memory, mongo, command, demo_accounts, feature_plan, element_picked, undo_point.
GET /__agentforge/api/projects	[[name, title, mtime, file_count, stack, unfinished, deployed:{state,target} null]]
GET /__agentforge/api/files/{project}	{ "<relative path>": {content, size} } — priority files first, capped at 120 files and 256 KB each
GET /__agentforge/api/models	{cloud[], local_models[], local[], cloud_enabled, cloud_via: 'api-key' 'signed-in' 'none', host, local_ctx}
GET POST /__agentforge/api/settings	GET returns ollama_host, cloud_enabled, api_key_hint, local_num_ctx, agent_model, image settings, mongodb_uri_set/hint, mongo status, deploy summary. POST accepts a patch of any of those plus srs_model, deploy_model, aws_*, vercel_token, deploy_mongodb_uri; secrets are stored and never echoed.
POST /__agentforge/api/jobs · GET /__agentforge/api/jobs/{job_id}	{method, path, body} → {job_id, status:'running'}; poll → {status:'running' 'done' 'error' 'unknown', http_status, result, error, elapsed}. Only /image and /logo-prompt may be run as jobs (_JOB_PATHS).
POST /__agentforge/api/image	{prompt, name?, project?, aspect, seed, force} → {ok, file, name, data_uri, url}. 409 when image generation is switched off, 503 when no Foocus answers.
POST /__agentforge/api/image-upload	{filename, data_base64, name?, project?} → the same body as /image. Needs no Foocus — that is the point.
POST /__agentforge/api/attach	{project, filename, data_base64} → {ok, text, ...}. A picture is saved into the project and described; a PDF and a recording come back as text, capped at ATTACH_TEXT_CAP with a note when trimmed.
POST /__agentforge/api/image-swap	{project, filename, data_base64, element} → {ok:true} immediately; the work runs in a thread and reports over the websocket like every other edit.
POST /__agentforge/api/undo	{project, id?} → {ok, restored:[paths], id}. Omitting id restores the most recent snapshot.
GET /__agentforge/api/qa/{project} · GET /__agentforge/api/qa-pdf/{project}	{project, have:{vitest,manifest,report,history,performance,tests}, vitest, manifest, report, history, performance, tests}. The PDF is rendered on request, not stored.
GET /__agentforge/api/srs-results/{project} · GET /__agentforge/api/srs-pdf/{project}	{project, have:{document,plan,handoff,interview,diagrams,pdf}, link, document, plan, handoff, interview, diagrams:[{name,mermaid,svg,png}]}
ANY /__agentforge/api/srs/* → http://127.0.0.1:7826/*	Streaming proxy with no read deadline (timeout=(2, None)) — a read timeout here cuts an interview off mid-answer. Returns 503 with `srs_status()` when the agent never imported.
SRS agent (:7826) project routes	POST /projects · GET /projects · GET POST /projects/{id} · POST /{id}/analyze · GET /{id}/questions · GET /{id}/interview · POST /{id}/interview/answer · POST GET /{id}/plan · POST /{id}/plan/approve · POST /{id}/generate-srs · GET /{id}/srs-json · /requirements · /diagrams · /ambiguities · /risks · POST /{id}/customize · POST /{id}/approve · GET /{id}/builder-handoff · GET /{id}/builder-prompt · GET /{id}/download/json pdf · GET /{id}/events · /traces · /errors · /events/stream · POST /{id}/inputs · /inputs-json · POST /jobs · GET /jobs/{id} · GET /health

Name	Shape
GET POST /__agentforge/api/deploy/{path} → http://127.0.0.1:7834/api/{path}	Same no-read-deadline proxy. `/deploy/runs` becomes `/api/runs`, so the studio never has to say `/deploy/api/...`.
POST /__agentforge/api/deploy/jobs · GET /__agentforge/api/deploy/jobs/{id}	Same shim shape as /api/jobs, targeting the deployment agent. `/jobs` paths are refused — jobs cannot run jobs.
POST /__agentforge/api/deploy-start	{project, target:'vercel' 'aws_ec2' 'aws_ecs', mongodb_uri?, validate_container?} → the run record. Rejects a missing production Mongo URI, a loopback one, an AWS target with no connected profile, and a second run for a project already deploying.
Deployment agent (:7834)	GET /api/health · /api/onboarding/status · /api/runs · /api/runs/{id} · /api/runs/{id}/events?after=N · /artifacts · /diff · /artifact?path= · /monitor · /evidence · /evidence/file?id= · /export.zip · /report.pdf · /domains. POST /api/onboarding/login · /api/mongodb/check · /api/aws/* · /api/runs/analyze (requires cloud_consent) · /api/runs/{id}/{action}. States: DRAFT, ANALYZING, REVIEW_READY, BOOTSTRAPPING, CI_RUNNING, DEPLOYING, VALIDATING, LIVE, FAILED, ROLLED_BACK, DESTROYED, CANCELLED.
Everything not under /__agentforge → http://127.0.0.1:5173	Byte-for-byte proxy of the generated app, including a raw socket relay for its HMR websocket. When the dev server is down, a document request gets a self-reloading "Preview not running" page and an asset request gets an immediate 502.

8. Repositories

AgentForge itself, and the branch it keeps each component on. Each component branch holds only that subsystem's paths, so it can be read as one thing.

Branch	What is on it
main	The shared runtime: server.py, the studio shell, the launchers.
srs-review-and-revision	The working branch where the whole system is developed.
srs-agent	srs-agent/ and studio/components/srs/ — the interview and the SRS.
builder-agent	agents/ — planning, generation, repair and the editing tools.
qa-agent	qa_agent/ and studio/components/testing/ — tests, e2e, the report.
deployment-agent	deployment-agent/ and studio/components/deploy/ — CI, AWS and Vercel.

Open source it is built on

Every third-party project the system depends on at runtime, and what each one is used for here.

Project	Repository	Used here for
Next.js	vercel/next.js	The studio, and every application AgentForge generates.
React	facebook/react	The studio's UI, and the generated apps' components.
Tailwind CSS	tailwindlabs/tailwindcss	Styling, in the studio and in generated apps.
Zustand	pmndrs/zustand	The studio's client state.
Lucide	lucide-icons/lucide	Icons throughout the studio.
JSZip	Stuk/jszip	Packaging a generated project for download.
FastAPI	fastapi/fastapi	The SRS agent's HTTP surface.
Uvicorn	encode/uvicorn	Serves the SRS app inside AgentForge's process.
Pydantic	pydantic/pydantic	Request and document schemas in the SRS agent.
LangGraph	langchain-ai/langgraph	The SRS agent's node graph: intake, classify, interview, plan, generate.
LangChain Core	langchain-ai/langchain	Message and tool primitives used by the graph.
Motor / PyMongo	mongodb/motor	Async and sync MongoDB access.
MongoDB	mongodb/mongo	Storage for SRS projects, and the database every generated app uses.
Ollama	ollama/ollama	Runs the language and vision models, locally or against a cloud host.
Playwright	microsoft/playwright	Screenshots for the pencil tool, and the end-to-end browser run.
Vitest	vitest-dev/vitest	The unit test runner the QA agent writes for and drives.
Lighthouse	GoogleChrome/lighthouse	The performance, accessibility, best-practice and SEO pass.
ReportLab	MrBitBucket/reportlab-mirror	Every PDF in the product, including this one.
pypdf	py-pdf/pypdf	Reading the text layer of an attached PDF.
PyMuPDF	pymupdf/PyMuPDF	Rasterising a scanned PDF page so the vision model can read it.

Project	Repository	Used here for
Pillow	python-pillow/Pillow	Image decoding, re-encoding and bounding for uploads.
Tesseract / pytesseract	tesseract-ocr/tesseract	Exact text out of an attached picture, alongside the model's reading.
faster-whisper	SYSTRAN/faster-whisper	Transcribes an attached or recorded voice note.
Foocus	llyasviel/Foocus	Generates the pictures and logos a generated app asks for.
boto3	boto/boto3	AWS: CloudFormation, EC2, ECS, and the credential chain.
Vercel CLI	vercel/vercel	The other deployment target.
websockets	python-websockets/websocket s	The live log, phases and streaming file writes into the studio.
watchdog	gorakhargosh/watchdog	Watches generated projects on disk.
jsonschema	python-jsonschema/jsonschem a	Validates deployment manifests.
PyYAML	yaml/pyyaml	Reads and writes the generated CI workflows.

Repository addresses are github.com/<slug> for each row above.